Re-implementing XMSS for Verification: A Project Summary

Wrenna Robson*

April 2026

Abstract

This report summarises a multi-implementation re-engineering of the XMSS and XMSS-MT stateful hash-based signature schemes (RFC 8391, including Errata 7900). Two implementations were produced from scratch: a C99 reference deliberately written to a restrictive subset that mirrors the constraints of the Jasmin language, and a Jasmin implementation covering five SHA-2/256 parameter sets (three single-tree and two multi-tree) with constant-time guarantees verified by the `jasmin-ct` checker on x86-64. The report profiles the RISC-V ISA surface of both implementations, develops a scoping analysis for a future Jasmin RISC-V port, and presents a candid account of currently known limitations. RISC-V ISA-specific optimisation work for XMSS is concentrated almost entirely in the hash layer: the algorithm layer compiles to portable RV64I + M and benefits negligibly from any extension beyond it.

1 Context and goals

XMSS is a stateful hash-based signature scheme standardised in RFC 8391 and approved by NIST in SP 800-208 as a post-quantum algorithm for stateful use cases such as firmware and code signing. Its security reduces to assumptions on the underlying hash function alone, and, unlike lattice schemes, it does not depend on algebraic structure that may be amenable to future quantum or classical cryptanalysis. The trade-off is statefulness: each secret key may sign only a fixed number of messages, and the signing state must be persisted and incremented atomically across signing operations.

This project pursues two goals that interact:

1. Produce an XMSS implementation suitable for formal verification of constant-time properties and (eventually) functional correctness, in a language whose semantics support such proofs. The chosen target is Jasmin, a restricted assembly-like language whose compiler preserves a constant-time discipline checkable at the source level by the `jasmin-ct` tool, and whose semantics are formalised in EasyCrypt.
2. Inform a future RISC-V 64 port. The Jasmin x86-64 backend is mature; the RISC-V backend is under active development. We want to know, before committing to that port, what ISA surface XMSS exercises and where work on the backend (or on architecture-specific code generation) would pay off.

The C implementation is not a competitor to the upstream reference maintained by the RFC authors; its purpose is to serve as a Jasmin-shaped specification. Every choice that diverges

*With Claude (Anthropic), which co-authored the implementations, analysis tooling, and report text. The work presented here is intended for evaluation and review and is *not* approved for production use.

from the reference is motivated by what the corresponding Jasmin code must look like, given Jasmin's prohibition of variable-length arrays, heap allocation, function pointers, and recursion. The result is a C codebase whose structure—module boundaries, calling conventions, buffer sizes, address-serialisation discipline—is a direct preview of the Jasmin code, and against which the Jasmin code can be cross-validated byte for byte.

2 Scope of the implementations

2.1 C99 reference

The C implementation supports all twelve XMSS and thirty-two XMSS-MT parameter sets registered in RFC 8391, across SHA-2/256, SHA-2/512, SHAKE-128, and SHAKE-256 hash backends. It uses the BDS authentication path algorithm [1] for amortised $O(1)$ signing cost.

The library is allocation-free: every buffer is either stack-local or caller-provided. The implementation respects an enforced subset that prohibits VLAs, function pointers, heap allocation, recursion, and any secret-dependent control flow or memory access. These constraints are tighter than RFC 8391 requires, and tighter than the upstream reference observes; they exist solely to make the code structurally portable to Jasmin.

Test coverage spans unit tests of every module, sign/verify roundtrips for every supported parameter set, BDS state serialisation, XMSS-MT tree boundary crossings, KAT cross-validation against the upstream reference, ACVP cross-checks, and exhaustive sign/verify across every leaf index for valid small-height parameter combinations.

2.2 Jasmin implementation

The Jasmin implementation covers five parameter sets at SHA-2/256: three single-tree and two multi-tree. Each parameter set is exposed as its own compiled assembly module with libjade-style export naming. Keygen, sign, and verify are tested for every set; the multi-tree sets are tested through tree boundary crossings (more than 1024 signatures).

The implementation passes the `jasmin-ct` constant-time checker on x86-64 across all twenty-four exported entry points. A small number of declassification sites are present and individually justified; the principal one is the leaf index, which is secret until written into the signature and public thereafter.

Cross-implementation interop tests sign with both the C and Jasmin backends from the same seed and compare both signatures and BDS state byte-for-byte after every signature, accounting for the endianness difference between the C wire-format big-endian state words and the Jasmin native little-endian internal representation.

3 Re-implementing XMSS in C: structural choices

The C implementation makes several design choices that visibly inflate its instruction count relative to the upstream reference. These are deliberate preparation for the Jasmin port. The following module-level deltas (measured on RV64 with `-O3`; full data in the comparative ISA report) are representative:

None of these deltas reflects any property of the XMSS algorithm. The reference's terseness comes from VLAs, contiguous caller-allocated hash-input buffers, and 32-bit-word ADRS structures

Table 1: Selected per-module instruction count differences between this project's C implementation and the upstream reference, with the design reason for each.

Module	Delta	Source of the difference
Parameter derivation	−791	Compact OID lookup table replacing a switch cascade (a Jasmin-neutral C-level optimisation).
Hash wrappers	+398	ADRS pre-serialised to a stack buffer and passed by pointer, matching the Jasmin calling convention.
WOTS+	+262	Defensive chain-length bound checks and the ADRS-by-pointer convention.
SHAKE module	+458	Incremental *init / absorb / finalize / squeeze* API, instead of a single caller-allocated whole-message buffer; necessitated by the no-VLA rule.

serialised inside each hash wrapper, all incompatible with the Jasmin subset. The headline gap of approximately 1942 instructions between the two archives is almost exactly the size of this project's bundled SHA-2 implementation (1953 instructions); the upstream reference delegates SHA-256 and SHA-512 to OpenSSL's `libcrypto` and so contains no SHA-2 internals at all. Once SHA-2 is netted out, the two archives agree to within eleven instructions in total size.

4 Re-implementing XMSS in Jasmin

The Jasmin language, by design, does not let one write the same kind of C that the upstream reference is written in. There is no heap allocation, no recursion, no `void *`, no function pointers, and, as became apparent only during implementation, no way at all to obtain a register-typed pointer to a stack-local variable. Several patterns the C implementation takes for granted required architectural workarounds.

4.1 The caller-provided scratch convention

The most consequential workaround is the *caller-provided scratch buffer*. The address structure (ADRS) is held as eight 32-bit words on the stack and serialised to thirty-two bytes for hashing. In C, that buffer is a stack-local. In Jasmin, the inability to take the address of a stack-local as a register pointer forces every function that needs ADRS bytes to receive a pointer to scratch space from its caller. This propagates up the call graph: every algorithm-layer function takes an explicit scratch pointer, and the top-level export functions take a 2240-byte caller-allocated scratch buffer.

Once adopted consistently, the convention eliminates any hidden allocation and makes stack frames predictable, properties that Jasmin code review and (eventually) EasyCrypt proofs benefit from.

4.2 Compile-time parameter specialisation

Jasmin has no const pointers and no inline parameter structs. Each parameter set therefore compiles to its own specialised assembly module, with parameter constants (N, W, H, D, chain length, index width, and so on) declared in a thin top-level wrapper file that includes the shared algorithm sources. Adding a new parameter set is, in the present design, on the order of fifteen lines of constant declarations and a new wrapper, with no algorithm-layer change.

This monomorphic compilation is also a testing advantage. Each Jasmin test exercises exactly one parameter combination through the production code path, with no chance of a parameter being silently substituted at runtime.

4.3 Constant-time discipline

The `jasmin-ct` checker verifies, at the source level, that no value annotated as secret reaches a branch condition or a memory address. The implementation passes the checker on all twenty-four export functions across all five parameter sets. The checker is strict in ways that traditional C analysers are not: it rejects, for example, a comparison whose result is later used to index a table, even if the table is constant.

A small number of declassification sites are needed. The principal one is the leaf index used during signing: it is secret until written into the signature, public thereafter, and the exhaustion check is a branch. WOTS+ chain lengths are derived from the message hash, which is public, and so do not raise CT obligations for the variable-trip chain loops.

The constant-time guarantee is a source-level property of the Jasmin program, preserved by the Jasmin compiler across backends. It will therefore carry to RISC-V, once the Jasmin RISC-V backend matures, without any reverification of the algorithm-layer code.

5 RISC-V ISA analysis

To characterise the ISA surface XMSS exercises on RISC-V, we profiled both this project's C implementation and the upstream reference. Both were cross-compiled with `riscv64-linux-gnu-gcc` 13.3.0 at `-O3` for both `rv64gc` (the standard general-purpose profile) and `rv64gc_zbb` (the same profile with the Zbb bit-manipulation extension enabled). Every instruction in each archive was classified by mnemonic against a lookup table generated from the `riscv-opcodes` database. Object-file granularity was preserved so that hash-layer and algorithm-layer instruction counts could be separated.

5.1 Required ISA: I + M is sufficient

Both implementations, compiled for the standard `rv64gc` profile, use only the **I** and **M** extensions. Table 2 gives the headline counts.

Table 2: Headline instruction counts with `-march=rv64gc -O3`.

Metric	This project's C	Upstream reference	Difference
Object files	13	9	−4
Total instructions	9562	7620	−1942
I	9505 (99.40%)	7541 (98.96%)	−1964
M	57 (0.60%)	79 (1.04%)	+22
A, F, D, Zb*	0	0	—
C-encoded ratio	48%	53%	+5 pp
Unique mnemonics	52	54	+2

The remaining 0.6% (1.04% for the reference) are M-extension multiply/divide instructions, all compiler-generated from ordinary arithmetic on array indices and parameter values. No A, F, D, Zba, Zbb, Zbs, Zicsr, or Zifencei instructions are emitted by either codebase under `rv64gc`. The

minimum functional ISA target for XMSS is therefore **rv64im**. The C extension (compressed encoding) halves average instruction width (around half of each archive uses 16-bit encoding) but adds no semantic capability and is invisible to source-level Jasmin code in any case.

The composition of the M-extension uses also differs between implementations (Table 3). The reference's larger share comes mostly from inline byte-offset multiplications inside its single combined core file and from divide/remainder operations inside its bundled Keccak module's variable-length absorb/squeeze loops. Neither pattern indicates structural dependence on M: an RV64I-only target would compile both codebases unchanged with shift-and-add expansions in place of the multiplies.

Table 3: M-extension mnemonic breakdown.

Mnemonic	This project	Reference	Compiler emits it for
`mulw`	37	68	32-bit index × parameter
`mul`	17	6	64-bit offset / pointer arithmetic
`divuw`	3	1	parameter derivation
`divu`	0	2	SHAKE absorb/squeeze offset (reference only)
`remu`	0	2	SHAKE block-length modulus (reference only)
Total	57	79	

5.2 Per-module breakdown

The two codebases carve XMSS into modules differently. The reference bundles WOTS+ public-key generation, L-tree, treehash, BDS state management, and XMSS / XMSS-MT signing into a single core file; this project splits those concerns across separate compilation units. Table 4 shows representative per-module counts. The reference's hash module is conspicuously small (684 instructions with no M uses) because it is a thin shim that delegates SHA-256 and SHA-512 to OpenSSL's `libcrypto`; SHA-2 internals are not present in the analysed reference archive at all.

Table 4: Per-module instruction counts under `-march=rv64gc`. Modules are grouped by role; the reference's combined core file is shown alongside this project's split modules.

Role / module	This project	Reference
Hash layer		
SHA-2 core	1953	— (in `libcrypto`)
Keccak / SHAKE	1337	1259
Hash wrappers	1082	684
Algorithm layer		
WOTS+	760	498
L-tree + treehash + BDS runtime	1743	$\subset$ 3169
BDS serialisation	473	— (not present)
XMSS / XMSS-MT core	1782	3682
ADRS manipulation	99	29
Parameters	286	1077
Utilities	47	29
Total	9562	7620

The headline 1942-instruction archive-size gap is almost entirely the size of this project's bundled SHA-2 implementation (1953 instructions). Once SHA-2 is netted out, the two archives agree

to within eleven instructions in total size. The remaining per-module deltas—hash wrappers, ADRS, WOTS+, parameters—are the design-tax items already discussed in Table 1.

5.3 Where Zbb lands

Re-compiling with `-march=rv64gc_zbb` introduces 164 Zbb instructions in this project's archive (132 in the reference's), as shown in Table 5. Both archives shed base-integer instructions (Zbb collapses multi-instruction rotation, $\overline{a} \wedge b$, and branch-based min/max sequences into single ops); the reference saves more in absolute terms because its baseline is smaller and its compositional structure exposes more recognisable idioms (see §5.4).

Table 5: Effect of enabling Zbb (`-march=rv64gc_zbb`).

	This project's C		Reference	
	rv64gc	rv64gc_zbb	rv64gc	rv64gc_zbb
I	9505	9281	7541	6982
M	57	57	79	79
Zbb	0	164	0	132
Total	9562	9502	7620	7193
Net change		−60		−427

The mnemonic mix (Table 6) is structurally asymmetric. This project's archive sees substantial 32-bit SHA-2 rotations (`rolw`, `roriw`) and byte-reversals (`rev8`); the reference's archive sees almost none, because its SHA-2 implementation is not in the archive (delegated to `libcrypto`). Conversely, the reference shows more 64-bit Keccak rotations (`rol`, `rori`) and significantly more `andn` uses, reflecting the prominence of the Keccak χ step in its bundled SHAKE module.

Table 6: Zbb mnemonic mix with `-march=rv64gc_zbb`.

Mnemonic	This project	Reference	Replaces	Where
`rolw`	41	0	3 insns	32-bit SHA-2 rotation
`roriw`	17	0	3 insns	32-bit SHA-2 rotation
`rev8`	19	0	multi-insn byte-swap	SHA-2 endianness
`rol`	17	38	3 insns	64-bit Keccak rho
`rori`	27	20	3 insns	64-bit Keccak rho
`andn`	28	51	2 insns	Keccak chi, masks
`maxu`	12	20	branch-based	BDS height comparisons
`minu`	3	2	branch-based	BDS / SHAKE min
`zext.h`	0	1	2 insns	16-bit widening
Total	164	132		

Both implementations show the same qualitative pattern: **Zbb uptake concentrates almost entirely in the hash layer.** In this project's archive 142 of 164 Zbb instructions (87%) land inside SHA-2 and SHAKE; in the reference, 108 of 132 (82%) land inside its bundled Keccak. The handful of Zbb uses that escape to the algorithm layer (around 22–24 in each implementation) are opportunistic `maxu`/`minu`/`andn` applications for index comparisons and bitmask combinations. None of these is structural: the algorithm runs equivalently without them.

5.4 Why the absolute Zbb savings are asymmetric

The two implementations *benefit* from Zbb to very different degrees: this project's archive nets 60 instructions saved; the reference nets 427. Two effects combine. First, this project's archive contains 32-bit SHA-2 rotations and byte-reversals that the reference does not, but those instructions replace three-instruction software sequences with a single instruction, so the proportional saving is bounded. Second, this project's SHAKE module is larger than the reference's by approximately 78 instructions because of its incremental context-based API (*init / absorb / finalize / squeeze*, required by the no-VLA rule), whereas the reference uses a single contiguous-buffer call shape; this raises the baseline against which the absolute saving is measured. A reference build that included SHA-2 in source form would show a substantially larger absolute Zbb benefit, dominated by the SHA-2 compression core's rotations and byte-reversals.

5.5 Implication for ISA-tuned code generation

The implication for any future ISA-tuning work—whether on the Jasmin RISC-V backend, on hand-written assembly intrinsics, or on the choice of pre-built primitive libraries—is that the payoff sits in the hash layer. The algorithm layer (WOTS+, L-tree, BDS, treehash, XMSS, XMSS-MT) is load–store–branch–shift code with no structural dependence on any extension beyond I + M, and ISA-specific work there will not move performance.

6 Scoping a future Jasmin RISC-V port

The ISA findings above bear on a port-scoping question: when a Jasmin RISC-V port becomes feasible, should it cover the whole stack, or only the hash primitives?

The default assumption is that the existing Jasmin algorithm-layer code should be recompiled for RISC-V. This path has appeal: the constant-time properties already established on x86-64 are source-level Jasmin properties and would carry to RISC-V without re-verification, and no new algorithm-layer work would be required.

The ISA evidence complicates the picture. Three observations bear on the decision:

1. Zbb uptake concentrates in the hash layer ($\geq 82\%$ in both codebases). The algorithm layer derives no ISA-specific benefit from a Jasmin port.
2. Upper-layer code-size differences between this project's C and the upstream reference are entirely Jasmin-portability tax (Table 1). A Jasmin algorithm layer pays this tax in source size and compilation overhead without buying any RV64-specific advantage.
3. Algorithm-layer code is ISA-neutral. A C or Rust upper layer over a Jasmin hash primitive would compile to substantially similar instruction streams.

The split-layer alternative this evidence suggests is to keep Jasmin for the hash primitives (where it earns formally checked constant-time *and* ISA-specific code generation) but write the algorithm layer in a less-restricted language: C, or, more attractively, Rust, which adds memory-safety guarantees that the algorithm layer cannot otherwise express. The precise integration shape (header-based FFI, language bindings, build-time composition) is an open question and is not prescribed here.

The alternative gives up three things:

- **End-to-end constant-time guarantees through Jasmin.** The `jasmin-ct` property is a source-level property of Jasmin code; once control crosses an FFI boundary into a different language, any CT property of the calling code has to be argued separately. The loss is

bounded: the algorithm layer's CT obligations (WOTS+ chain index, BDS array indexing) are simple enough to audit by inspection or with tools targeting the host language, and the hard CT cases (variable-time instructions inside hash rounds) all live in the hash layer, which remains under Jasmin.

- **The finished Jasmin upper layer.** The existing Jasmin algorithm-layer code is complete, tested, CT-verified, and cross-validated against the C reference. Replacing it would discard a finished body of work; this is a real cost even if the forward case for the split-layer architecture is strong.
- **Symmetry with a future formal-verification effort.** If the project later pursues a full EasyCrypt-style correctness proof, that proof infrastructure is for Jasmin code; an upper layer in another language puts an EasyCrypt proof of the upper layer out of reach. No such proof effort is currently underway.

This section records the evidence rather than recommending a path; the decision needs both the ISA argument and the integrity arguments side by side.

7 Known limitations and open work

Both implementations are functionally complete within their declared scope, but several known limitations remain. At the time of writing, **no security-affecting defects remain outstanding.** Four security items previously identified during review—secret-material zeroing across hash and signing paths; reliance on volatile semantics in constant-time comparison and memory-zeroing utilities; missing bounds validation on the BDS retain parameter; and toolchain permission hardening—have all been resolved during the work covered here. The limitations that remain fall into three categories.

7.1 Functional and specification issues

- **Jasmin verify does not validate the OID embedded in the public key**, an explicit deviation from the Jasmin implementation's own specification document, which states that verify must check it. The omission is partially mitigated by Jasmin's compile-time parameter specialisation: a public key with a mismatched OID would already fail verification at the root-comparison step, so the security impact is a confusing error message rather than an unsound accept. The C implementation does perform the check.
- **XMSS and XMSS-MT OID namespaces overlap.** RFC 8391 assigns separate OID registries for XMSS and XMSS-MT; the value `0x00000001` is XMSS-SHA2_10_256 in one registry and XMSSMT-SHA2_20/2_256 in the other. A caller who passes a public key into the wrong verify entry point with a parameter struct that happens to share the same OID value will get a spurious OID-check pass and a downstream signature-shape failure. This is an RFC design limitation rather than an implementation bug; the OID check on its own does not distinguish XMSS from XMSS-MT keys.

7.2 Coverage gaps in the test suite

- Cross-implementation deep interop is currently exercised on the multi-tree 20/2 parameter set but not on 20/4. The 20/4 set is exercised by the C exhaustive boundary-crossing test and by the Jasmin API roundtrip, but there is no per-signature BDS-state comparison between implementations for it.
- The Jasmin implementation lacks an explicit key-exhaustion test (signing through the entire 2^H index space and confirming the documented refusal at exhaustion). The behaviour

is exercised indirectly by the C implementation's exhaustive tests but should be verified directly on the Jasmin side.
- The C unit-test suite does not cover the L-tree and treehash modules in isolation. Both are exercised by every higher-level test, but a targeted unit test would isolate regressions.
- The C test suite's coverage of the SHAKE backend and of the $n = 64$ parameter sets is thinner than its SHA-2/256 coverage.
- The XMSS-MT unit test only spot-checks four of the 1025 signatures around a tree boundary; full per-signature coverage of the boundary would catch a wider class of state-machine regressions.

7.3 Code-quality and ergonomic issues

A number of cleanup and ergonomic items are tracked: a wasteful dual hash-context declaration in one C hash-message wrapper; absence of a signature-length parameter on verify, which would allow callers to detect truncated inputs; subtle BDS swap logic in XMSS-MT sign that would benefit from documentation; a moderate stack-pressure concern (a 34 KB stack-allocated swap temporary in XMSS-MT sign, which is fine on hosted targets but undesirable for embedded use); and helper-function duplication across Jasmin test files.

7.4 Scope work that has not been started

Three substantive pieces of work are explicitly out of scope for the implementations as they stand:

- **Hash backends beyond SHA-2/256 in Jasmin.** SHA-2/512 and SHAKE-128/256 backends are designed but not written; completing them would extend Jasmin coverage to the remaining parameter sets registered in RFC 8391.
- **Code review of the Jasmin sources.** A full review pass against the Jasmin specification document is open.
- **Formal verification beyond constant-time.** The EasyCrypt proof effort for functional correctness has not begun.

8 Methodological observations

A few observations from the implementation process transfer to the next verification-oriented re-implementation effort and are not peculiar to XMSS.

Specifications are most useful when they are written independently of the code. The Jasmin implementation's specification document was written from the RFC and from declared design decisions, deliberately without reference to the Jasmin source. This makes it a check on the implementation rather than a description of it. The verify-side OID-check deviation described in the previous section was identified by a routine spec-versus-code reading.

Stateful-scheme test discipline must be exhaustive at small parameter sizes. The natural failure mode of a stateful authentication-path scheme is a state-machine bug that manifests only at specific index boundaries and only on traversals that cross them. Spot-check tests that sign one or two messages and verify them are insufficient to distinguish "the signing function works" from "the BDS state update preserves the invariants required for the next signature."

Exhaustive sign/verify across every leaf index for valid small-height parameter combinations is tractable and has caught concrete bugs in this project that wider but shallower test coverage missed. Cross-implementation interop should compare internal BDS state byte-for-byte after every signature, not just the signature outputs.

Test each layer in isolation before building on it. The Jasmin implementation grew bottom-up: address structure, utilities, SHA-256, WOTS+, L-tree, treehash, BDS, XMSS, XMSS-MT, each tested against the C output before the next layer was written. The top-level XMSS export functions assembled with no algorithmic bugs, because every component had already been validated. By contrast, attempts to write multiple top-level functions in one pass before compiling produced tangled register-pressure failures that took considerably longer to unbraid.

Reach for runtime tools first when the failure is a runtime symptom. A non-determinism in early Jasmin keygen output was diagnosed in seconds by a memory-error detector. The same bug was opaque to manual inspection of the surrounding register-spill-heavy assembly. Code review is the right tool for compile-time errors and design issues; for wrong output, non-determinism, or crashes, the runtime tool should come first.

A previous claim that "X passes" is unverified until the test that demonstrates it is named. Several incidents in this project trace back to summary claims of correctness that, on inspection, were not backed by tests covering the parameter combination in question. The corollary is the test discipline now in place: every parameter set is signed through at least one full tree boundary, and any claim of correctness names the test that demonstrates it.

9 Summary of conclusions

Two complete, cross-validated implementations.
A C99 reference covering all RFC 8391 parameter sets and a Jasmin implementation covering five SHA-2/256 parameter sets pass exhaustive cross-validation against the upstream reference and against each other. The Jasmin implementation is verified constant-time on x86-64 by `jasmin-ct`.

XMSS uses only RV64I + M.
Both implementations compile to base integer plus a small handful of compiler-emitted multiply instructions on `rv64gc`. The minimum functional ISA target is `rv64im`. No floating-point, atomic, or bit-manipulation extensions are required.

Zbb lives in the hash layer.
Where the Zbb extension is enabled, $\geq 82\%$ of the additional Zbb instructions land in SHA-2 / SHAKE / Keccak. The algorithm layer benefits negligibly. Any ISA-tuning effort for XMSS—compiler, hand-written assembly, or backend development—should target the hash primitives.

The algorithm layer is design-tax-bound, not algorithm-bound.
Module-level instruction count differences between this project's C and the upstream reference are explained entirely by Jasmin-portability constraints. The XMSS algorithm itself is small.

The upper-layer scoping question is open.
The existing Jasmin upper layer is finished and tested; the ISA evidence suggests its

constraints buy little RV64-specific benefit. A split-layer alternative (Jasmin hash primitives, Rust or C upper layer) is plausible; the trade-offs are discussed in §6 without prescribing a choice.

No security-affecting defects currently outstanding.
Four security items identified during review have all been resolved. Remaining limitations are concentrated in testing-coverage gaps, code-quality items, and unfinished scope (hash backends beyond SHA-2/256 in Jasmin). One functional deviation is documented: Jasmin verify does not validate the public-key OID, though compile-time parameter specialisation means a mismatched OID would still fail verification at the root-comparison step.

References

[1] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. *XMSS—A practical forward secure signature scheme based on minimal security assumptions.* In Post-Quantum Cryptography (PQCrypto 2011), LNCS 7071, 2011.